applied sciences

MDPI

Article

Payload-Independent Continual Detection of DDoS and Low-Rate Attacks in Software-Defined Networks Using SCX

Shijin Liu [1], Hiroaki Fukuda [1,*] and Paul Leger [2,*]

[1] Department of Computer Science and Engineering, Shibaura Institute of Technology, Tokyo 135-8548, Japan; ma22160@shibaura-it.ac.jp
[2] Escuela de Ingeniería, Universidad Católica del Norte, Coquimbo 1780000, Chile
* Correspondence: hiroaki@shibaura-it.ac.jp (H.F.); pleger@ucn.cl (P.L.)

Abstract

Software-defined networking (SDN) improves network programmability by separating network control from packet forwarding. However, distributed denial-of-service (DDoS) and low-rate attacks can exhaust network and service resources through different traffic patterns. A detection model needs to identify these patterns and use incoming traffic when new labels are limited. This paper studies self-organizing map (SOM)-guided Continuous XGBoost (SCX) for payload-independent detection of DDoS and low-rate attacks. First, a self-organizing map is trained with the initial labeled data and assigns pseudo-labels to incoming flow records. Next, a filtering rule combines neuron label agreement, sample support, and quantization error to select records for training. The accepted records are added to the previous training data, and XGBoost is retrained to update the detector. The input features are obtained from packet headers, packet lengths, timestamps, and flow counters, without inspecting application contents. We describe the implementation, derive the sample-acceptance conditions, and analyze the time and memory requirements. The main evaluation averages twenty cases per dataset, randomly selected from 201 candidate partitions on InSDN, CIC 2017 DDoS, and CIC 2017 LR-DDoS. After ten updates, mean accuracy is 99.785%, 99.923%, and 99.543%, with total initial-to-final gains of 0.121, 0.032, and 0.523 percentage points. Corresponding mean F1 scores are 99.770%, 99.937%, and 99.394%. The low-rate improvement is mainly associated with higher recall, alongside a small precision decrease. Supplementary controlled ablations examine the reliability–coverage trade-off. These results characterize the update trajectories within fixed classification tasks; adaptation to new attacks, natural concept drift, and end-to-end SDN mitigation remain unevaluated.

Keywords: software-defined networking; DDoS detection; low-rate attacks; continual learning; semi-supervised learning; self-organizing map; XGBoost; payload-independent detection

Check for updates

Academic Editor: Hernán Astudillo

Received: 13 September 2026
Revised: 10 October 2026
Accepted: 13 October 2026
Published: 14 October 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Software-defined networking (SDN) separates the control plane from the data plane and allows forwarding policies to be managed through software. OpenFlow provides an interface through which a controller can install and modify forwarding rules [1]. This design improves network programmability and supports centralized policy management [2]. It also changes how a defense system collects traffic information and applies a response. The controller, switches, and monitoring links must share the workload of detection and mitigation [3].

Distributed denial-of-service (DDoS) attacks threaten this architecture by consuming communication capacity, processing time, or connection state. High-rate attacks usually generate large traffic volumes, whereas low-rate (LR) DDoS attacks can interrupt a service with short bursts or persistent connections. The Shrew attack shows how periodic bursts can interfere with TCP recovery [4]. Other low-rate attacks exploit connection behavior and service resources. SDN detection studies have therefore considered flow interactions and temporal features in addition to packet rates [5,6]. A detector that considers only traffic volume may miss these differences.

The information required by the detector is another important design choice. Self-organizing map (SOM)-guided Continuous XGBoost (SCX) uses network/transport headers, packet lengths, arrival times, and aggregated flow counters rather than application contents. The removed identifiers and constant fields are not payload features; related flow-based studies also use non-payload measurements [7,8]. An exporter attached to an SDN switch or mirrored data-plane interface can collect these observations without sending application contents to the controller. Bidirectional and active/idle statistics still require per-flow state beyond ordinary OpenFlow counters. The experiments assess exported records, not an in-switch classifier or a deployed mitigation pipeline [2,3].

Machine learning provides a way to combine these features, but obtaining labeled traffic remains a practical challenge. A model can be initialized with labeled records while later traffic arrives without annotations. Semi-supervised learning uses the structure of this unlabeled data to support model training [9]. Continuous learning also requires a method for adding new information while retaining useful information from earlier data [10,11]. In SDN security, online ensemble methods have already been investigated for this purpose [12].

Among tabular models, XGBoost is attractive for heterogeneous flow features because gradient-boosted trees capture nonlinear interactions with a favorable accuracy–cost balance [13,14]. However, XGBoost is supervised and normally requires reliable labels. Real LR-DDoS monitoring provides few labeled attack samples but continuously produces unlabeled flows, so an adaptive detector must exploit this traffic without allowing noisy pseudo-labels to damage the decision boundary.

This paper studies SOM-guided Continuous XGBoost (SCX) for DDoS and low-rate attack detection. The SOM–XGBoost backbone, purity/inverse-distance confidence, batch-distance filtering, and cumulative retraining build on Liu et al. [15]. This work extends evaluation to InSDN and Slowloris/Slowhttptest, specifies the support penalty and implementation conventions, derives admission conditions and complexity, and evaluates label reliability under a common initial budget. These additions extend the analysis and evaluated attack coverage; they do not establish adaptation to unseen attack families.

The basic procedure has two phases. During initialization, a small labeled collection is used to construct the SOM and train XGBoost. During continuous learning, the SOM assigns a candidate label to each incoming flow. A confidence rule combines neuron label agreement, the number of supporting samples, and quantization error. Samples that pass this rule and a batch-distance condition are added to the training data. XGBoost is then retrained with the initial and accumulated data. The SOM remains fixed while the classifier is updated.

The objective of this paper is to alleviate detection-performance limitations caused by insufficient labeled data for XGBoost, using SOM-filtered incoming unlabeled flows for successive retraining. Here, “continual” refers to continued training within an existing classification task; adaptation to new attack families and concept drift are outside the present scope and require separate investigation. Addressing label scarcity remains valuable even

without such adaptation, because additional traffic can contribute to classifier training without requiring new manual annotations.

This study focuses on payload-independent traffic features, pseudo-label filtering for continuous learning, and detection performance over successive model updates. The analysis identifies the required packet and flow measurements, derives the sample-admission conditions, and evaluates changes in accuracy, precision, recall, F1, and false-positive rate across updates.

The main contributions of this study are as follows:

1. We describe a payload-independent input model based on packet-header metadata and flow measurements. The description identifies the measurement state needed to obtain each feature family.
2. We formulate the SCX processing steps, including neuron labeling, confidence calculation, sample filtering, data accumulation, and classifier retraining. We also derive the acceptance-radius condition and analyze computational complexity.
3. We evaluate eleven classifier stages on InSDN, CIC 2017 DDoS, and CIC 2017 LR-DDoS. Twenty cases per dataset describe the mean initial-to-final changes and the intermediate behavior of five detection metrics.
4. We discuss the effects of flow export, processing time, model updates, and training-data storage on the integration of SCX into an SDN defense system.

2. Background and Detection Requirements

2.1. SDN Traffic Monitoring and Response

An SDN switch forwards packets according to installed rules, while the controller manages forwarding policies and processes network events. Traffic can be measured at a switch, collected by a flow exporter, or analyzed by a separate service. The placement of these components affects both processing overhead and communication cost [2,3]. Even a fast classifier can create a large monitoring load if detailed records are exported too frequently.

The defense process can be divided into measurement, detection, and response. Measurement converts observed traffic into features. Detection assigns a class to those features. Response applies a network policy based on the detection result. In an OpenFlow-based system, these functions can run in different components [1]. For example, a flow exporter maintains traffic counters, a classifier processes the exported records, and the controller installs the required rules. The experiments in this paper evaluate the classification and update procedures after numerical flow records are available.

2.2. DDoS and Low-Rate Attack Behavior

High-rate and low-rate attacks place different demands on a detector. Flooding attacks can produce high packet rates, large byte volumes, or many short flows. Low-rate attacks can instead produce long connections, periodic bursts, or an imbalance between connection state and transferred data. SDN studies have used combinations of flow-rule attributes and traffic features to detect these behaviors [5,6,16].

Low-rate attacks also include different mechanisms. Shrew attacks use timed bursts to affect TCP retransmission [4]. Slowloris and Slowhttptest are slow service-exhaustion scenarios included in the CIC 2017 resource [17]. Flow duration, inter-arrival times, packet lengths, and directional counters provide different descriptions of their activity. The information contained in a record depends on how the exporter groups packets and ends a flow.

A single feature is therefore insufficient to describe every attack. A high packet rate can indicate flooding, but it provides less information about a persistent slow connection. Similarly, a long flow can belong to either normal communication or an attack. Feature-

engineering studies show the importance of selecting and combining traffic attributes for SDN detection [18,19]. In SCX, these attributes have two roles: they determine SOM distances and provide the input variables for XGBoost.

2.3. Continuous Learning with Unlabeled Traffic

The initial labeled data provides the starting point for the detector. New traffic is then processed in batches without using its benchmark labels for training. The system must generate candidate labels and decide which samples can be added to the training set. Semi-supervised learning uses assumptions about data structure to make this decision [9]. The number of available samples alone does not determine their usefulness.

SCX uses the labeled SOM as the reference for pseudo-label generation. A sample receives higher confidence when it is close to a neuron supported by consistent initial labels. The map controls the samples entering the training set, while XGBoost provides the current classification result. This procedure uses local map information instead of relying only on the classifier's prediction probability.

The fixed map in SOM also determines which regions can contribute new training data. A sample far from the initial map may fail the filtering rule even if it belongs to a new traffic pattern. Thus, the initial representation remains important throughout continuous learning. The classifier can change after retraining, but the sample-labeling reference does not change in the evaluated implementation.

Concept drift refers to changes in the data-generating process over time [20]. Continual-learning studies also examine how learning later tasks affects performance on earlier tasks [11]. The present experiments use successive batches within each traffic collection. Each experiment retains its initial labeled data and evaluates all model versions on the same classification task.

2.4. Detection Requirements

The following requirements guide the SCX workflow. First, the input features must have a clear source in packet headers, packet lengths, timestamps, or flow aggregation. Second, the system needs an explicit rule for accepting pseudo-labeled samples. Third, the fixed components must be separated from the data and classifier that change during learning. Finally, the evaluation must consider attack detection and false alarms together. Accuracy, precision, recall, F1, and false-positive rate describe different parts of binary detection, especially when the class proportions differ [21].

These requirements also guide the implementation. The feature schema connects the exporter to the model. The filtering rule controls training-data intake. The update procedure determines when a new classifier replaces the old one. Per-stage measurements then show how each replacement affects detection performance.

3. Related Work

3.1. Flow-Based Monitoring and Feature Selection

Flow-based detection uses aggregated traffic measurements to identify abnormal network behavior. In SDN, these measurements can support both centralized decisions and distributed monitoring. Wang et al. proposed a source-based defense that combines sFlow with SOM [22]. Liu et al. developed an LSSOM-based real-time defense system [23]. These studies connect map-based traffic analysis with monitoring and response procedures.

Several studies focus on selecting the traffic features used by the classifier. Polat et al. evaluated feature-selection methods with machine-learning models for SDN DDoS detection [8]. El Sayed et al. studied flow-based anomaly detection with feature selection, while Liu et al. investigated feature engineering for SDN detection [7,18]. Han et al.

proposed XRDI-based feature selection and evaluated the resulting inputs with several classifiers [19]. These results show that the input representation is an important part of the detection method.

Other recent approaches explicitly use payload bytes. Kiflay et al. combine flow statistics with the first 32 bytes of transport-layer payload for intrusion detection [24]. Boulaiche et al. construct CNN inputs from byte-frequency images of session payloads [25]. For broader traffic classification, YaTC represents five packets using up to 80 header bytes and 240 payload bytes per packet [26]. SCX instead uses header metadata and packet/flow measurements for both labeling and classification, avoiding payload-byte collection as a model-input requirement. This distinction concerns the monitoring inputs, rather than a matched performance comparison.

SCX also processes numerical flow features. However, its features are used by two connected components. The SOM uses standardized distances to assign and filter samples, and XGBoost uses the accepted samples to train the classifier. Removing a field can therefore change both pseudo-label selection and classification. Section 4.2 describes the retained feature families and their measurement requirements.

3.2. Deep Learning and Low-Rate Detection

Deep-learning methods provide another approach to DDoS detection. LUCID uses a lightweight convolutional model and considers the computational requirements of traffic classification [27]. Elubeyd and Yiltas-Kaplan proposed a hybrid deep-learning method for DoS/DDoS detection in SDN [28]. Nawaz et al. investigated a developed deep neural network for detection and mitigation [29]. Mehmood et al. later evaluated an optimizer-equipped CNN–MLP model [30]. These methods mainly improve the feature representation or classifier. SCX focuses on using unlabeled records in subsequent classifier training.

Wang et al. proposed a Multi-Dimensional Deep Convolutional Classifier (MDDCC) that combines wavelet decomposition and convolutional neural networks for SDN attack detection [31]. Their workflow uses switch-port statistics for initial detection and learned traffic features for detailed classification. SCX instead uses a fixed SOM to select pseudo-labeled flow records for subsequent classifier updates.

Low-rate detection requires additional attention to sparse and persistent traffic. Pérez-Díaz et al. proposed a flexible SDN architecture that uses machine learning for low-rate identification and mitigation [5]. Wu et al. used a factorization machine to learn interactions among flow-rule features [6]. Liu et al. studied an RF-based low-rate detection system with selected features [16]. These methods combine several traffic measurements because average traffic intensity does not fully describe a low-rate attack.

Widodo et al. combined CatBoost-based recursive feature elimination with an MLP for low-rate attack detection using a subset extracted from CICIDS2017 [32]. They selected a seven-feature configuration with a low false-positive rate. This feature-selection approach complements the SCX focus on selecting incoming records for further training.

More recently, DELP-Net combined a differentiable entropy representation with a convolutional pyramid and temporal modeling [33]. Its evaluation uses windows covering six low-rate attack types. A window-based model can describe changes across a sequence of observations. SCX instead classifies individual exported flow records, whose features summarize the packets within each flow. The two approaches use different observation intervals to represent low-rate behavior.

3.3. Online and Continual Learning

Online learning updates the classifier as new observations arrive. Adaptive random forests use ensemble mechanisms for evolving data streams [34]. ADWIN detects changes

through a variable-length observation window [35]. Gama et al. reviewed concept-drift methods and the relation between distribution changes and model adaptation [20]. For SDN security, Alashhab et al. proposed an online ensemble with dynamic feature selection and a detection and mitigation procedure [12].

Continual learning studies both the acquisition of new information and the retention of previous knowledge. Parisi et al. and De Lange et al. reviewed strategies such as replay and constraints on model updates [10,11]. Gradient Episodic Memory uses stored examples in a task-oriented learning procedure and provides measures of transfer between tasks [36]. These studies motivate the separation of continued training from task-retention evaluation.

Nguyen et al. combined a modified one-dimensional AlexNet with Spatial Location Constraint Prototype Loss (SLCPL) and fuzzy C-means clustering to identify unfamiliar attack samples [37]. Network experts label suspicious samples before incremental learning. Their evaluation includes the Friday DDoS collection from CICIDS2017, providing an update-stage result alongside the pseudo-label-based updates studied here.

SCX primarily addresses the issue of insufficient labeled data. SCX retains its initial labeled data and accumulates accepted pseudo-labeled records. Each update trains a new XGBoost model with this growing collection. The SOM remains fixed, and the batch sequence determines when the update is performed. The resulting procedure is batch-continuous self-training with cumulative replay. This paper evaluates the classifier after each batch within the three datasets.

Label-limited DDoS detection is an established research direction. LEDEM uses semi-supervised detection within an SDN–cloud IoT architecture [38], and Hallaji et al. compare thirteen semi-supervised algorithms under partial labeling and class imbalance [39]. The present contribution is the analysis and controlled evaluation of the specified SCX admission procedure, rather than the first use of unlabeled traffic for DDoS detection.

3.4. Position of This Study

Monitoring methods address measurement locations and collection procedures, feature-selection methods determine the input, and online learning defines how new observations enter training. An SDN detector combines these choices with an update and response procedure.

This paper connects SCX with a payload-independent feature description, a mathematical analysis of sample filtering, and per-stage detection results. Data preparation is also important to this evaluation. Previous studies have examined extraction and labeling issues in the CIC resources [40,41]. Security machine-learning guidelines further emphasize clear partitioning and evaluation procedures [42]. We therefore describe the data preparation and experimental configuration together with the results.

4. SCX Detection Method

4.1. Problem Definition and System Workflow

A flow record is represented by a feature vector $\mathbf{x} \in \mathbb{R}^d$ and a label $y \in \{0, 1\}$. The labels 0 and 1 represent normal and attack traffic, respectively. The system first receives an initial labeled dataset $\mathcal{D}_0$. It then processes unlabeled batches $\mathcal{B}_1, \ldots, \mathcal{B}_T$. A fixed test dataset $\mathcal{H}$ is used to evaluate the model after initialization and each update. The benchmark labels of the stream records are not used to generate pseudo-labels or train the updated classifier.

Figure 1 shows the SCX workflow. First, the initial data is used to fit a scaler, train the SOM, label its neurons, and train XGBoost. Next, the SOM generates candidate labels and confidence scores for incoming samples. Samples that pass the filtering conditions are added to a training buffer. XGBoost is retrained with this buffer, and the new model

replaces the previous classifier. Traffic detection uses the current XGBoost model and the same feature transformation.

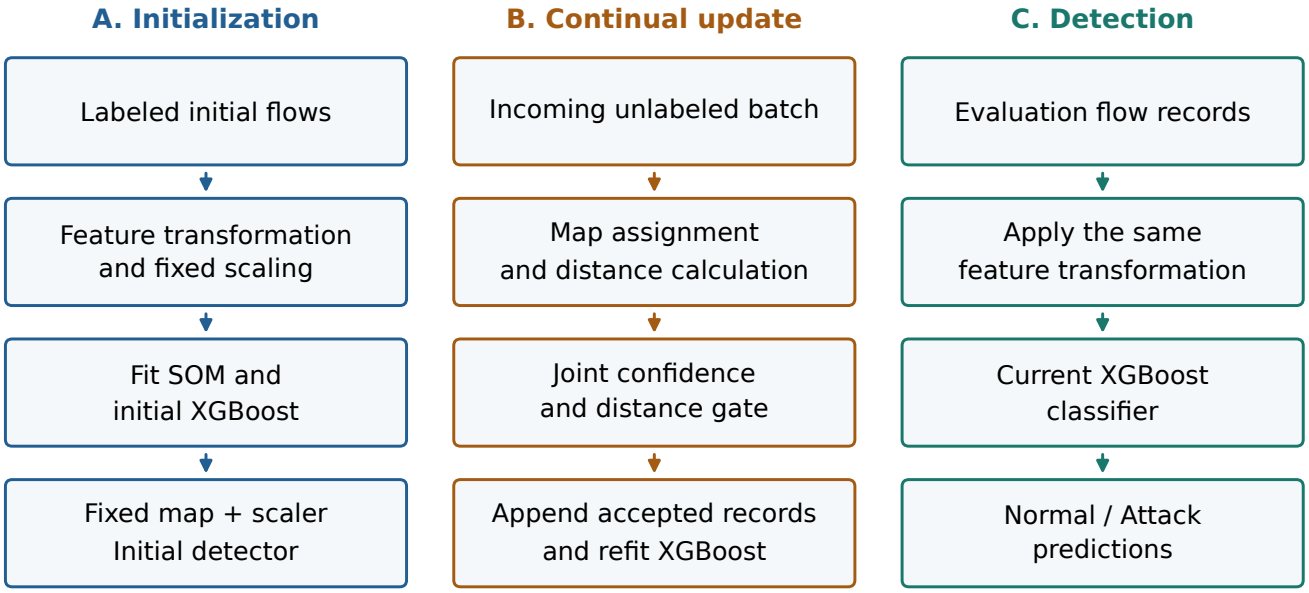


Figure 1. SCX workflow.

Table 1 lists the main symbols used in this section. The current accepted batch contains only the samples selected at the current update. The replay buffer contains the samples retained from all processed batches.

Table 1. Principal notation.

Symbol	Meaning
d, T	Number of input features and update batches
$\mathcal{D}_0$, $\mathcal{H}$	Initial labeled collection and fixed test collection
$\mathcal{B}_t$, $\mathcal{Z}_t$	Raw and standardized observations in batch t
$\mathbf{w}_j$, $\mathcal{V}$	Weight vector of neuron j and set of neurons
$b(\mathbf{z})$, $q(\mathbf{z})$	Best matching neuron and quantization error
n_j, ℓ_j, $\widetilde{\rho}_j$	Initial support, assigned label, and agreement score
$c(\mathbf{z})$, τ	Candidate confidence and admission threshold
Q_p, $q_t^\star$	Empirical quantile operator and batch distance cutoff
$\mathcal{A}_t$, $\mathcal{C}_t$	Current accepted batch and cumulative replay cache
$\mathcal{D}_t$, F_t	Fitting collection and classifier at stage t

4.2. Payload-Independent Input Features

SCX uses packet and byte counts, packet-length statistics, flow duration, rates, inter-arrival times, TCP flags, header lengths, transport-window information, and active/idle statistics. Protocol and retained port fields are obtained from network or transport headers. The input does not contain payload bytes, HTTP message contents, or DNS names. The CIC 2017 resource provides flow statistics extracted from traffic captures [17].

Table 2 shows how these features are measured. Packet length can be read for each packet, while its flow mean requires an accumulated total and a packet count. Inter-arrival time requires timestamps from successive packets. Forward and backward statistics require a consistent direction rule. Therefore, SCX does not require application contents, but its flow features require measurement state across packets.

Table 2. Feature provenance and observation requirements.

Feature Family	Observable Basis	Required Measurement State
Header metadata	Protocol, flags, header lengths, transport window, retained ports	Parsing of network/transport headers
Volume and size	Packet lengths, packet counts, byte totals, directional summaries	Per-flow counts and accumulated statistics
Timing and rates	Arrival times, duration, intervals, packets/bytes per unit time	Timestamps and consistent time units
Active and idle behavior	Gaps and activity periods within a flow	Exporter-specific activity thresholds
Directional structure	Forward/backward totals and summaries	Stable flow key and direction convention

For InSDN, we remove Flow ID, source IP, source port, destination IP, destination port, and timestamp, together with fourteen constant features. The remaining input contains 63 features. CIC 2017 DDoS uses 77 features. For CIC 2017 LR-DDoS, Protocol is removed before feature preprocessing, leaving 77 features. After whitespace in the column names is removed, the ordered names match those in CIC 2017 DDoS. The scaler, SOM, and XGBoost model are trained separately for each dataset.

After feature preparation, standardization is defined by Equation (1)

$$z_k = \frac{x_k - \mu_k}{s_k}, \qquad s_k = \begin{cases} \sigma_k, & \sigma_k > 0, \\ 1, & \sigma_k = 0, \end{cases} \tag{1}$$

where μ_k and σ_k are the mean and population standard deviation of feature k in the initial labeled dataset. A scale of one is used when the standard deviation is zero. The fitted scaler is retained for all batches so that SOM distances use the same scale. Section 6.2 describes the preceding encoding and missing-value processing.

4.3. Map Assignment and Neuron Labels

The SOM organizes observations using a topology-structured set of weight vectors [43]. For a standardized observation $\mathbf{z}$, the best matching unit (BMU) and quantization error are

$$b(\mathbf{z}) = \arg\min_{j \in \mathcal{V}} \|\mathbf{z} - \mathbf{w}_j\|_2, \tag{2}$$

$$q(\mathbf{z}) = \|\mathbf{z} - \mathbf{w}_{b(\mathbf{z})}\|_2. \tag{3}$$

Equation (2) finds the neuron closest to the input sample. Equation (3) measures the corresponding distance. Both calculations use the standardized feature vector.

Let $n_{j,k}$ be the number of initial labeled records of class k assigned to neuron j, and let $n_j = \sum_{k=0}^{1} n_{j,k}$. For an occupied neuron, its label and empirical purity are defined in Equation (4)

$$\ell_j = \arg\max_{k \in \{0,1\}} n_{j,k}, \qquad \rho_j = \frac{\max_{k \in \{0,1\}} n_{j,k}}{n_j}, \qquad n_j > 0. \tag{4}$$

The implementation uses a fixed vote order to resolve ties. Purity measures the proportion of the majority class at a neuron, while support measures the number of mapped

samples. A neuron with few samples can have high purity, so SCX considers both quantities when calculating confidence.

An empty neuron receives the label of a nearest already labeled grid location. This process follows the implementation's traversal order, so a previously propagated label can also be used. The neuron receives a fallback score $\rho_\emptyset$. Thus, $\widetilde{\rho}_j = \rho_j$ for occupied neurons and $\widetilde{\rho}_j = \rho_\emptyset$ for empty neurons. The fallback score is an assigned value rather than purity measured from labeled samples.

4.4. Support-Aware Confidence and Joint Admission

SCX applies the support penalty in Equation (5)

$$r(n) = \begin{cases} \eta, & n < n_{\min}, \\ 1, & n \geq n_{\min}, \end{cases} \qquad 0 < \eta < 1, \tag{5}$$

and assigns candidate confidence using Equation (6)

$$c(\mathbf{z}) = \frac{r(n_{b(\mathbf{z})})\,\widetilde{\rho}_{b(\mathbf{z})}}{1 + \lambda q(\mathbf{z})}, \qquad \lambda > 0. \tag{6}$$

The numerator combines neuron label agreement with the support penalty. The denominator reduces confidence as quantization error increases. A sample receives a high score when its neuron has consistent labels, sufficient support, and a nearby weight vector. The score is a filtering measure rather than a calibrated class probability.

For the standardized batch $\mathcal{Z}_t$, define its distance cutoff as in Equation (7)

$$q_t^\star = Q_p\big(\{q(\mathbf{z}) : \mathbf{z} \in \mathcal{Z}_t\}\big), \qquad 0 < p < 1, \tag{7}$$

where Q_p is the empirical quantile at level p. The accepted records are defined by Equation (8)

$$\mathcal{A}_t = \Big\{(\mathbf{z}, \ell_{b(\mathbf{z})}) : \mathbf{z} \in \mathcal{Z}_t,\ c(\mathbf{z}) \geq \tau,\ q(\mathbf{z}) \leq q_t^\star\Big\}, \qquad 0 < \tau \leq 1. \tag{8}$$

A sample must pass both conditions. The confidence threshold checks its compatibility with the fixed map. The quantile threshold removes samples with relatively large distances in the current batch. The accepted proportion also depends on the confidence scores and on ties at the quantile threshold.

The filtering rule uses the initial map, its neuron statistics, and the current unlabeled batch. Each accepted sample keeps its neuron's label as the training target. The implementation includes a sample-count limit, but the limit is not reached in these experiments. Equation (8) therefore describes the filtering procedure used for the reported results.

4.5. Training-Data Accumulation and XGBoost Retraining

Let $\mathcal{C}_0$ be empty and let $\mathcal{D}_0^z$ denote the standardized initial labeled collection. The replay state evolves according to Equation (9)

$$\mathcal{C}_t = \mathcal{C}_{t-1} \oplus \mathcal{A}_t, \qquad \mathcal{D}_t = \mathcal{D}_0^z \oplus \mathcal{C}_t, \tag{9}$$

where $\oplus$ represents record concatenation. Each accepted batch is added once. Every training set contains the initial labeled data and all previously accepted samples, allowing earlier records to participate in later updates.

Gradient boosting constructs an additive prediction function through successive function updates [44]. XGBoost implements a regularized tree-ensemble formulation and

efficient fitting procedures [13]. Using σ for the logistic function, its binary prediction can be written as in Equation (10)

$$\widehat{p}_t(\mathbf{z}) = \sigma(F_t(\mathbf{z})), \qquad F_t(\mathbf{z}) = \beta_t + \sum_{m=1}^{K_t} f_{t,m}(\mathbf{z}), \tag{10}$$

where β_t is an intercept and each $f_{t,m}$ is a tree contribution. A corresponding regularized objective is given in Equation (11)

$$\mathcal{L}_t = \sum_{(\mathbf{z},y)\in\mathcal{D}_t} [-y \log \widehat{p}_t(\mathbf{z}) - (1-y)\log(1-\widehat{p}_t(\mathbf{z}))] + \sum_{m=1}^{K_t} \Omega(f_{t,m}). \tag{11}$$

The loss is calculated over the accumulated training data. The regularization term controls the complexity of the decision trees.

If $\mathcal{A}_t$ is nonempty, SCX trains a new XGBoost model on $\mathcal{D}_t$ and replaces the current classifier. If the batch contains no accepted samples, the previous model is retained. Each retraining step builds a new ensemble instead of adding trees to the previous one. The scaler, map weights, and neuron labels remain fixed. Their complete settings are retained with the source code. Algorithm 1 summarizes the processing steps.

Algorithm 1: SCX initialization and batch-continuous detection

Input: Initial labeled collection $\mathcal{D}_0$; batches $\mathcal{B}_1, \ldots, \mathcal{B}_T$; test collection $\mathcal{H}$; gate configuration

Output: Classifier sequence $F_0, \ldots, F_T$ and per-stage metrics

Prepare numerical features according to the recorded schema;
Fit the scaler on the initial labeled partition and transform all collections;
Fit the SOM; assign neuron labels, support, and agreement scores;
Fit F_0 on $\mathcal{D}_0^z$; set $\mathcal{C}_0 \leftarrow \varnothing$;
Evaluate F_0 on the transformed test collection;
for $t \leftarrow 1$ **to** T **do**
    Compute $b(\mathbf{z})$, $q(\mathbf{z})$, and $c(\mathbf{z})$ for $\mathbf{z} \in \mathcal{Z}_t$;
    Compute $q_t^\star$ and form $\mathcal{A}_t$ using the joint gate;
    Set $\mathcal{C}_t \leftarrow \mathcal{C}_{t-1} \oplus \mathcal{A}_t$;
    if $\mathcal{A}_t \neq \varnothing$ **then**
        Fit a new F_t on $\mathcal{D}_0^z \oplus \mathcal{C}_t$;
    else
        Set $F_t \leftarrow F_{t-1}$;
    Evaluate F_t on the same transformed test collection;

After each update, the current model is evaluated on the same transformed test data. Test labels are used for scoring and are not included in the training targets. Section 6.4 describes the configuration and selection of the reported experimental runs.

5. Analytical Properties and Computational Requirements

5.1. Confidence Bounds and Distance Sensitivity

This section analyzes the confidence rule and its computational requirements. For neuron j, let $a_j = r(n_j)\widetilde{\rho}_j$. Both factors lie between zero and one, giving $0 \le a_j \le 1$. For a sample assigned to this neuron, Equation (12) gives the confidence bounds:

$$c_j(q) = \frac{a_j}{1+\lambda q}, \qquad 0 \le c_j(q) \le a_j \le 1. \tag{12}$$

For a fixed BMU, Equation (13) shows that confidence is non-increasing in quantization error and attains its maximum a_j at $q = 0$.

$$\frac{\partial c_j}{\partial q} = -\frac{\lambda a_j}{(1+\lambda q)^2} \leq 0. \tag{13}$$

Purity and support provide different information. Purity measures agreement among mapped labels. The support multiplier reduces the score when that agreement is based on few initial samples. Both values remain fixed after initialization. Consequently, receiving more unlabeled samples does not increase a neuron's reference confidence. The initial labeled data continues to determine which regions receive strong support.

The derivative applies within the region assigned to one neuron. When the BMU changes, the sample uses another neuron's support and agreement values. Confidence can therefore change at the boundary between two regions, even when their weight vectors are close.

5.2. Equivalent Acceptance Radius

For a candidate assigned to neuron j, the confidence condition can be rearranged as in Equation (14)

$$\frac{a_j}{1+\lambda q} \geq \tau \quad \Longleftrightarrow \quad q \leq R_j = \frac{a_j/\tau - 1}{\lambda}, \qquad a_j \geq \tau. \tag{14}$$

If $a_j < \tau$, no non-negative distance satisfies the confidence threshold. If $a_j = \tau$, only zero quantization error satisfies it. For an admissible neuron, the joint gate therefore imposes the effective radius in Equation (15)

$$R_{j,t}^{\mathrm{eff}} = \min\{R_j, q_t^\star\}. \tag{15}$$

The batch quantile can tighten the admissible radius but cannot enlarge it beyond the confidence-derived radius.

Let $\mathcal{V}_j^{\mathrm{B}}$ denote the set of standardized observations assigned to neuron j by the BMU rule, and let $\mathbb{B}(\mathbf{w}_j, r)$ denote a closed Euclidean ball. The accepted feature vectors in a batch belong to

$$\mathcal{Z}_t \cap \bigcup_{j:a_j \geq \tau} \left(\mathcal{V}_j^{\mathrm{B}} \cap \mathbb{B}(\mathbf{w}_j, R_{j,t}^{\mathrm{eff}}) \right). \tag{16}$$

Equation (16) describes the filtering rule geometrically. The SOM divides the feature space into BMU regions. Within an eligible region, SCX accepts only the samples that fall inside the effective radius.

The two thresholds have different effects. If an entire batch is far from the map, a sample must still satisfy the confidence-derived radius. If a batch is close to a well-supported neuron, the quantile condition can provide the tighter restriction. Thus, the final decision depends on both the fixed neuron statistics and the distance distribution of the current batch.

5.3. Growth of the Training Buffer

Write $A_t = |\mathcal{A}_t|$ and $n_0 = |\mathcal{D}_0|$. When the training cap is inactive, the fitting-collection size is given by Equation (17)

$$n_t = n_0 + \sum_{u=1}^{t} A_u. \tag{17}$$

Each accepted batch contributes once to this total. Later updates train on the accumulated buffer, so the initial data and earlier accepted records remain available when a new ensemble is built.

The proportion of initial labeled samples is n_0/n_t. The number of these samples remains unchanged, but their proportion decreases as new records are added. This matters in a long-running system because all records receive the same sample weight in the accumulated training set. Reweighting, a bounded buffer, or corrections from delayed labels would change how earlier data influences later models.

The SOM and classifier also change at different stages of the procedure. The SOM assignments remain fixed, but adding a batch changes the data used to train XGBoost. Two successive classifiers can therefore make different predictions. A region whose samples always fail the filtering rule contributes no new training records. These properties explain why the performance curve can fluctuate during updating: retraining minimizes the training objective rather than directly controlling the next test score.

5.4. Time and Memory Complexity

Let $M = |\mathcal{V}|$ be the number of neurons, d the feature dimension, and $B_t = |\mathcal{B}_t|$ the current batch size. A direct BMU search computes distances to M vectors for each observation, giving $O(B_t M d)$ time. Once the BMU and distance are available, confidence evaluation and threshold tests require $O(B_t)$ scalar operations. An empirical quantile can be obtained through selection or sorting; sorting gives a conservative $O(B_t \log B_t)$ bound.

Let $C_{\mathrm{XGB}}(n_t, d)$ denote the cost of fitting the configured XGBoost model on the current cache. For a nonempty accepted batch, the update cost can be summarized as in Equation (18)

$$C_t = O(B_t M d + B_t \log B_t + n_t d) + C_{\mathrm{XGB}}(n_t, d), \tag{18}$$

excluding test evaluation and data-loading overhead. The $n_t d$ term accounts for forming the dense accumulated training array in the implementation. XGBoost fitting is written separately because its cost depends on tree construction, sparsity, parallelism, and estimator settings [13].

During initialization, a direct SOM procedure with I vector updates requires $O(IMd)$ distance and weight-update operations. Feature preparation and initial classifier training add further cost. Section 7 reports initial classifier fitting time and recorded experiment execution time; SOM fitting and scoring are cached separately in these measurements.

With a materialized batch and dense replay cache, numerical storage scales as in Equation (19)

$$S_t = O((M + n_t + B_t)d) + S_{\mathrm{XGB},t}, \tag{19}$$

where $S_{\mathrm{XGB},t}$ includes the ensemble and its fitting workspace. Labels and neuron statistics require additional lower-dimensional arrays. The recorded process memory also includes loaded libraries, temporary copies, and runtime allocations. It is therefore larger than the storage occupied by the training buffer alone.

5.5. Implications for Update Scheduling

The batch size and buffer size affect different parts of the update. A larger current batch increases the work needed for SOM assignment. A larger accumulated buffer increases retraining cost. Longer update intervals reduce the frequency of retraining, but new observations must wait longer before they affect the classifier. Shorter intervals allow more frequent updates while repeatedly processing the accumulated data.

The experiments use ten update batches. A long-running service would also need rules for buffer growth, batches with few accepted samples, and changes in traffic distributions.

6. Experimental Design

6.1. Datasets and Detection Tasks

To evaluate SCX, we use InSDN, CIC 2017 DDoS, and CIC 2017 LR-DDoS. InSDN provides traffic collected in an SDN environment [45]. The two CIC collections are taken from the CIC 2017 resource introduced by Sharafaldin et al. [46]. They provide conventional DDoS and slow attack traffic. For each experiment, normal traffic is labeled 0 and the included attack categories are combined as label 1.

Table 3 shows the evaluated data. InSDN contains 68,424 normal records and 68,424 attack records. The attack class includes 47,527 DDoS and 20,897 DoS records. CIC 2017 DDoS contains 38,806 normal records and 61,194 DDoS records. CIC 2017 LR-DDoS contains 18,705 normal records and 11,295 attack records, including 5796 Slowloris and 5499 Slowhttptest records.

Table 3. Datasets used in the experiments.

Dataset	Records	Features	Normal	Attack
InSDN	136,848	63	68,424	68,424
CIC 2017 DDoS	100,000	77	38,806	61,194
CIC 2017 LR-DDoS	30,000	77	18,705	11,295

InSDN uses all of `InSDN_Balanced.csv`: Normal, DDoS, and DoS are filtered from `Normal_data.csv`, `metasploitable-2.csv`, and `OVS.csv`. All normal records are retained; attacks are proportionally downsampled without replacement (seed 42), preserving source order. Reconstruction matches the selected record hashes. CIC 2017 DDoS uses rows 0–99,999 of the local `Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv`; CIC 2017 LR-DDoS uses rows 0–29,999 of `lrddos.csv`. These contiguous prefixes are not row-wise selections by prediction outcome. The files contain 325,745 and 31,303 rows, respectively, so the selections do not exhaust them. Their complete assembly and capture ordering remain unverified.

6.2. Data Preparation and Partitioning

Infinities become missing values, and both medians and scaler statistics are fitted only on the initial labeled partition and frozen. All retained fields are numerical. InSDN, CIC 2017 DDoS, and CIC 2017 LR-DDoS contain 0/3/5 missing and 0/29/39 infinite cells, affecting 0/16/22 rows, respectively.

The main evaluation uses twenty selected group-disjoint random partitions per dataset. Nominal group proportions are 10% for initialization, with test proportions of 30% for InSDN and CIC 2017 LR-DDoS and 25% for CIC 2017 DDoS. Remaining groups form ten update batches. Table 4 reports the mean record counts; unequal group sizes produce varying record-level budgets. This random order mixes capture conditions rather than preserving chronology and evaluates batchwise self-training within an existing classification task.

Table 4. Mean record counts for initialization, updating, and testing.

Dataset	Initial Labeled	Unlabeled Stream	Fixed Test
InSDN	18,029.400	78,146.350	40,672.250
CIC 2017 DDoS	10,000.750	64,990.650	25,008.600
CIC 2017 LR-DDoS	2994.200	18,013.650	8992.150

Step 0 represents the initial model, and steps 1–10 represent the models obtained after successive updates. Each model is tested on the same records within its experiment. The

five performance curves therefore contain eleven observations. All stages are retained, including local decreases and unchanged values.

The selected collections contain 51,797/1237/1538 repeated feature vectors beyond first occurrences. Partitions keep each identical-feature group within one initial/stream/test role and one stream batch, without using labels to construct groups. Group proportions need not equal record proportions. This prevents exact-vector overlap, not all session dependence. Flow ID is not a certified session identifier, and local CIC session metadata are unavailable.

The CIC machine-learning CSV inputs used here lack absolute capture timestamps and flow identifiers. Flow duration and inter-arrival statistics do not recover the global capture order, so these files alone do not support a verified chronological split. Official CIC resources also include timestamped labeled flows and PCAPs [17]; rebuilding and linking those records is outside the present evaluation. InSDN has ambiguous short dates and separately captured scenarios [45]. Under either date interpretation, its earliest nominal 10%, retaining timestamp ties, contains 13,737 DoS records and no normal records; under day/month parsing, its latest nominal 30% is all normal. Sorting these tables therefore does not establish a historical two-class initialization. Verified capture metadata and a class-covered historical window are needed. The present evaluation concerns label-limited batchwise self-training, not natural drift or temporal continuity.

6.3. Performance Metrics

We evaluate accuracy, precision, recall, F1, and false-positive rate. True positives (TP) are attack records correctly classified as attacks, and true negatives (TN) are normal records correctly classified as normal. False positives (FP) are normal records classified as attacks, while false negatives (FN) are attacks classified as normal. For $N = TP + TN + FP + FN$, the metrics are defined in Equations (20)–(24)

$$\text{Accuracy} = \frac{TP + TN}{N}, \tag{20}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \tag{21}$$

$$\text{Recall} = \frac{TP}{TP + FN}, \tag{22}$$

$$\text{F1} = \frac{2TP}{2TP + FP + FN}, \tag{23}$$

$$\text{FPR} = \frac{FP}{FP + TN}. \tag{24}$$

Precision, recall, and F1 are calculated for the attack class. All denominators are nonzero in the reported evaluations. Metric values are multiplied by 100 and presented as percentages. Differences between percentage values are reported in percentage points (pp).

Accuracy measures the overall proportion of correct predictions. Precision measures the reliability of attack predictions, while recall measures the proportion of attacks detected. FPR measures false alarms among normal records. These metrics should be considered together because the three collections have different class proportions. Precision–recall analysis is particularly useful under unequal class frequencies [21].

Metrics are reported to three decimal places, with all calculations performed using unrounded values.

6.4. Configuration and Run Selection

Table 5 gives the fixed configuration. Vote ties use initial-record order. Occupied neurons follow first-BMU encounter order; empty neurons are traversed row-major and

inherit from the nearest already labeled neuron, including previously propagated empty neurons. Distance ties use insertion order.

Table 5. SCX configuration.

Setting	Value
SOM grid	20×20 for InSDN/DDoS; 15×15 for LR-DDoS
SOM initialization	MiniSom normalized uniform-random weights; no data/PCA weight initialization
SOM training	Gaussian rectangular neighborhood; Euclidean BMU; learning rate 0.500, $\sigma = 1.500$; default asymptotic decay; $\max(1000, 2n_0)$ random updates
Gate	$\tau = 0.850$ (InSDN), 0.900 (CIC); $\lambda = 2.000$, $n_{\min} = 5$, $\eta = 0.500$, $p = 0.850$; empty-neuron purity 0.100
XGBoost shared	10 trees; depth 6; learning rate 0.100; binary logistic objective; logloss; CPU histogram algorithm; one thread
Initial/update XGBoost	L_1: 0.000/0.100; L_2: 1.000/0.100; row and column sampling: 1.000/0.800; other defaults fixed by XGBoost 2.1.4
Updates and storage	10 batches; pseudo-label ratio limit 100 and fitting cap 300,000 are inactive at these sizes; no cache eviction
Control/ablation seeds	Same twenty splits as the main SCX runs
Numerical environment	Python 3.12.14; NumPy 2.0.2; pandas 2.2.3; SciPy 1.14.1; scikit-learn 1.6.1; MiniSom 2.3.0; XGBoost 2.1.4; River 0.22.0

For each dataset, twenty distinct partitions were randomly selected from the 201 candidates generated using the group-disjoint procedure in Section 6.2. Selection was independent of model performance, and no partition was excluded or replaced on the basis of accuracy, F1, initial-to-final gains, or update-trajectory shape. The selected partition seeds are listed below in ascending order. The main SCX experiments, classifier controls, and component ablations used the same twenty partitions for each dataset.

InSDN: 9, 22, 28, 36, 46, 53, 58, 59, 83, 95, 102, 125, 126, 135, 155, 162, 167, 172, 177, 193.
CIC 2017 DDoS: 16, 45, 46, 64, 68, 81, 86, 94, 97, 99, 102, 134, 138, 140, 141, 143, 152, 169, 170, 187.
CIC 2017 LR-DDoS: 2, 19, 27, 29, 30, 43, 50, 54, 56, 91, 93, 100, 103, 113, 124, 135, 146, 155, 167, 173.

6.5. Supplementary Controlled Repetition and Sensitivity Protocol

Classifier controls, component ablations, and admission diagnostics use the same twenty partitions per dataset as the main SCX results. Within each repetition, methods share the initial records, stream batches, features, and holdout. Exact-feature groups are class-stratified; preprocessing, SOM, and the initial classifier are refitted for each repetition. Cached initial-map scores are shared only by controlled variants. Stream truth is used retrospectively for diagnostics, not pseudo-label generation or training.

Controls are (i) the frozen initial XGBoost, (ii) an initial-set-only refit with the update configuration, and (iii) XGBoost-confidence self-training at the same numerical threshold. Ablations independently set purity to one, remove the low-support penalty, remove the inverse-distance factor, or remove the batch-quantile condition. The remaining conditions are unchanged.

The XRDI + DT adaptation [19] ranks features on the common initial set using the published normalized XGBoost gain-times-cover, RF/DT Gini importance, and empirical

information-gain combination, then freezes a top-ten-feature tree. It does not reproduce the original balancing and 70%-labeled protocol. The ARF adaptation [34] uses ten River trees, initial labels, and the same SOM pseudo-labels thereafter; drift feedback therefore uses pseudo-labels, not new ground truth. ADWIN [35] is a change detector rather than a classifier.

Main stage-wise means and sample standard deviations use the twenty selected runs with equal weights, not the eleven correlated stages within a run. Changes are calculated within each run before averaging. Final confusion matrices are row-normalized per run before averaging. Source records overlap across partitions, which are not independently collected networks.

Gate sensitivity uses three gate-validation pairs and a validation subset comprising 25% of the initial label budget. Eleven fixed one-factor settings vary τ by ± 0.050, $\lambda \in \{1, 2, 4\}$, $n_{\min} \in \{1, 5, 10\}$, $\eta \in \{0.250, 0.500, 1.000\}$, and $p \in \{0.500, 0.850, 1.000\}$. Initial-budget sensitivity instead uses the twenty main split seeds at 5%, 10%, and 20% of exact-feature groups, keeping each split's holdout fixed. These are nominal group fractions. Medians, scaling, and SOM fitting use only the initial set at each budget. The 10% results reuse the main runs; all additional outcomes are retained. These analyses do not select the main configuration.

6.6. Runtime Environment and Resource Measurement

Resource results use the saved summaries of the same twenty selected runs. Initial classifier fitting time is recorded separately. Recorded experiment execution time includes classifier initialization, ten updates, evaluation, diagnostic calculations, and output operations, but excludes separately cached SOM fitting and scoring. It is an offline software measurement, not batch latency or end-to-end detection delay.

Memory is converted from bytes using 1024^2 and reported in MiB. Each run samples process resident memory after classifier stages; its largest sample is recorded, and these maxima are averaged across the twenty runs. This is not a continuously monitored peak. Acquisition time and the delay required to form a flow record are discussed separately in Section 8.

7. Experimental Results

The main tables and curves consistently use the same twenty cases per dataset, as defined in Section 6.4. Each curve averages the corresponding classifier stage across these runs. Classifier controls and component ablations use the same partitions.

7.1. Initial and Final Detection Performance

Table 6 shows the mean initial and final performance of SCX across the twenty cases. Final accuracy reaches 99.785% on InSDN, 99.923% on CIC 2017 DDoS, and 99.543% on CIC 2017 LR-DDoS. The corresponding F1 scores are 99.770%, 99.937%, and 99.394%. Final accuracy and F1 exceed their initial values in all three selected-case averages.

Table 6. Initial and final SCX performance (mean ± SD, %).

Dataset	Stage	Accuracy	Precision	Recall	F1	FPR
InSDN	Initial	99.664 ± 0.088	99.650 ± 0.208	99.627 ± 0.271	99.638 ± 0.129	0.324 ± 0.186
	Final	99.785 ± 0.083	99.811 ± 0.162	99.729 ± 0.182	99.770 ± 0.104	0.183 ± 0.173
CIC 2017 DDoS	Initial	99.892 ± 0.019	99.924 ± 0.054	99.899 ± 0.052	99.911 ± 0.016	0.120 ± 0.085
	Final	99.923 ± 0.020	99.948 ± 0.031	99.926 ± 0.035	99.937 ± 0.016	0.082 ± 0.050
CIC 2017 LR-DDoS	Initial	99.021 ± 0.417	99.414 ± 0.248	97.982 ± 1.108	98.689 ± 0.556	0.349 ± 0.148
	Final	99.543 ± 0.184	99.381 ± 0.270	99.407 ± 0.302	99.394 ± 0.240	0.374 ± 0.164

InSDN precision and recall increase by 0.161 and 0.102 pp. CIC 2017 DDoS has smaller gains of 0.024 and 0.027 pp, with a 0.038 pp reduction in FPR. CIC 2017 LR-DDoS has the largest accuracy and F1 gains, at 0.523 and 0.704 pp. Its recall increases by 1.424 pp, while precision decreases by 0.033 pp and FPR increases by 0.025 pp.

7.2. Accuracy over Continuous-Learning Updates

Figure 2 shows mean accuracy over the initial stage and ten updates. For InSDN, accuracy increases from 99.664% to 99.719% after the first update. Small reversals occur at updates 2 and 5, followed by further gains to 99.785% at the final step. Every updated mean remains above the initial mean.

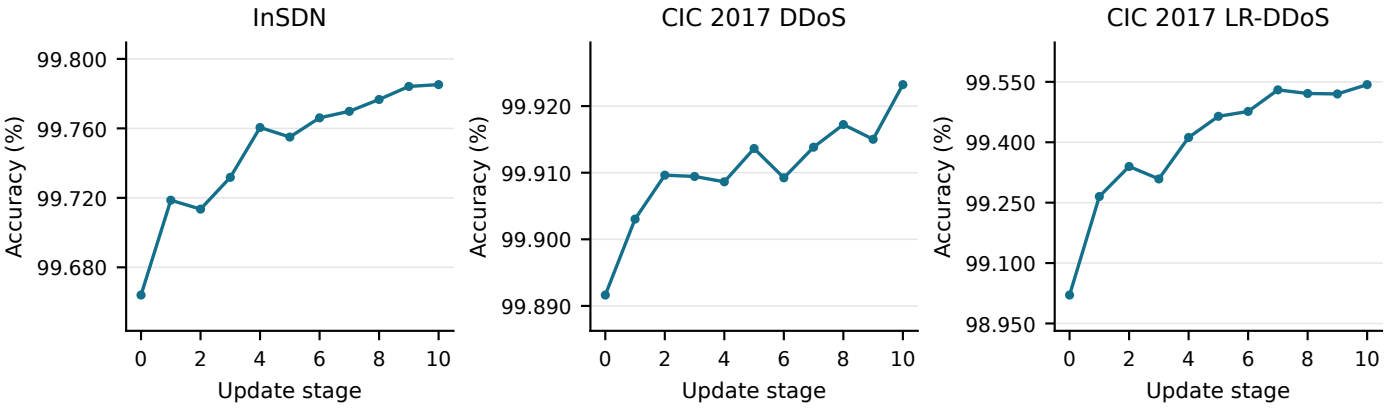


Figure 2. Mean accuracy over successive updates.

CIC 2017 DDoS changes within a narrower range. Mean accuracy increases from 99.892% to 99.903% after the first update. Later updates produce small gains and reversals, with the highest mean of 99.923% reached at the final step.

CIC 2017 LR-DDoS shows the largest overall rise. Mean accuracy increases from 99.021% to 99.265% at update 1 and 99.340% at update 2. After a small reversal at update 3, it reaches 99.530% at update 7 and finishes at 99.543%. The main gains are distributed across the early and middle updates.

7.3. Precision and False-Positive Rate

Figure 3 shows mean precision. InSDN changes little during the first two updates, then rises from its initial 99.650% to 99.811% at the final step. This later precision improvement accompanies the continued accuracy and F1 gains.

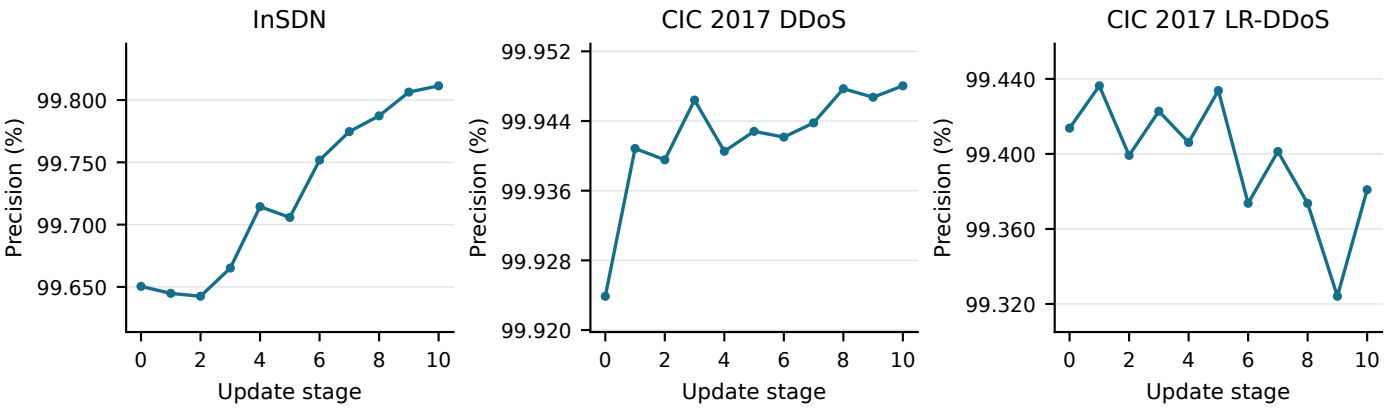


Figure 3. Mean attack-class precision over successive updates.

CIC 2017 DDoS precision remains in a narrow range, increasing from 99.924% to 99.948%. CIC 2017 LR-DDoS fluctuates around its initial 99.414%, reaches 99.324% at update 9, and recovers to 99.381% at update 10. Its final mean precision is slightly below the initial value despite the higher accuracy and F1.

Figure 4 shows mean false-positive rate from the normal-traffic perspective. InSDN decreases from 0.324% to 0.183%, while CIC 2017 DDoS decreases from 0.120% to 0.082%. CIC 2017 LR-DDoS increases from 0.349% to 0.374%, with its largest value of 0.409% at update 9. Its improved attack detection therefore accompanies a small increase in false alarms.

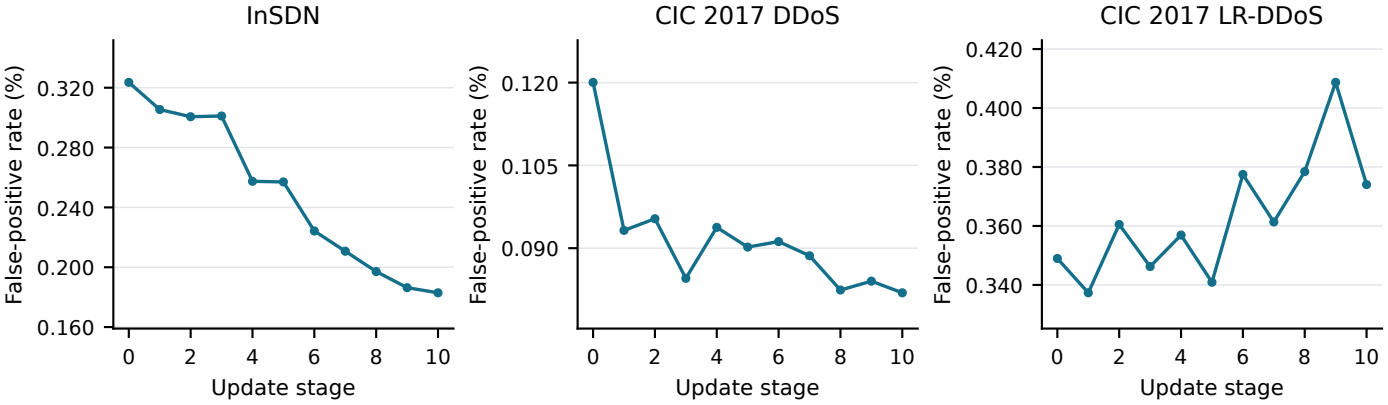


Figure 4. Mean false-positive rate over successive updates.

Precision and FPR describe related behavior with different denominators. Precision considers the records predicted as attacks. FPR considers the records whose true class is normal. For CIC 2017 LR-DDoS, the precision recovery from update 9 to update 10 coincides with a decrease in FPR, although both final means remain less favorable than their initial values.

7.4. Recall and F1 Score

Figure 5 shows mean attack-class recall. InSDN increases from 99.627% to 99.748% after the first update and peaks at 99.771% at update 4. It then declines slightly to 99.729%, still above initialization. Later precision gains compensate for this local recall reduction in the F1 trajectory.

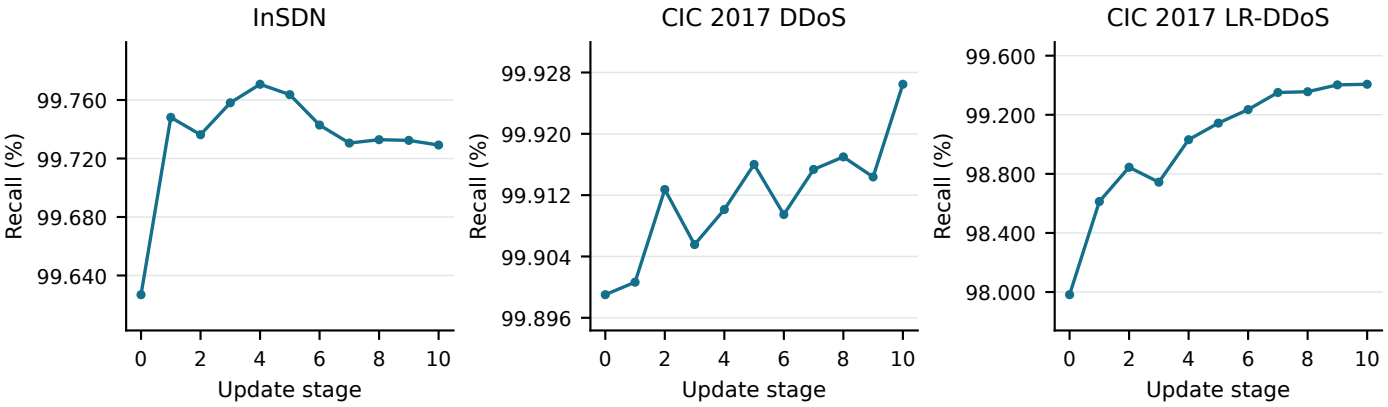


Figure 5. Mean attack-class recall over successive updates.

CIC 2017 DDoS recall varies within a small range. It increases from 99.899% to 99.926%, reaching its highest mean at the final step. Both precision and recall contribute to the small final F1 improvement.

CIC 2017 LR-DDoS recall increases from 97.982% to 98.613% at update 1 and 98.845% at update 2. After a small reversal at update 3, it continues upward to 99.407% at update 10. This 1.424 pp gain is larger than the accompanying precision reduction and is the main metric change associated with the higher F1.

Figure 6 shows the combined precision–recall performance. Mean F1 increases from 99.638% to 99.770% on InSDN, from 99.911% to 99.937% on CIC 2017 DDoS, and from 98.689% to 99.394% on CIC 2017 LR-DDoS. All three mean curves reach their highest value at update 10, with smaller intermediate reversals retained in the plots.

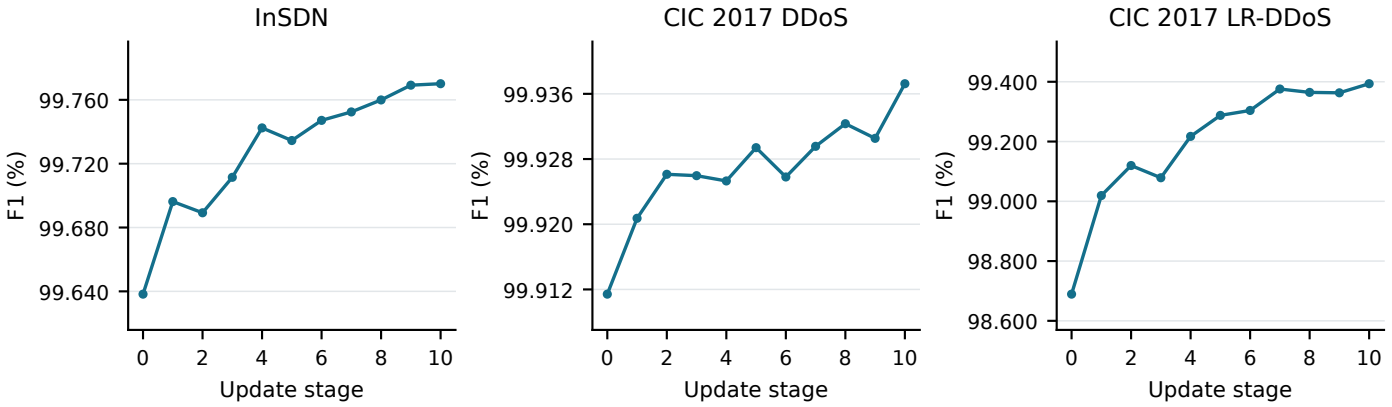


Figure 6. Mean attack-class F1 over successive updates.

7.5. Final Classification by Traffic Class

Figure 7 shows the mean final confusion matrices. Each run is first normalized by the number of records in each true class; the twenty matrices are then averaged with equal weights. The normal-to-normal cell represents mean specificity, and the attack-to-attack cell represents mean recall. The off-diagonal cells represent mean false-positive and false-negative rates.

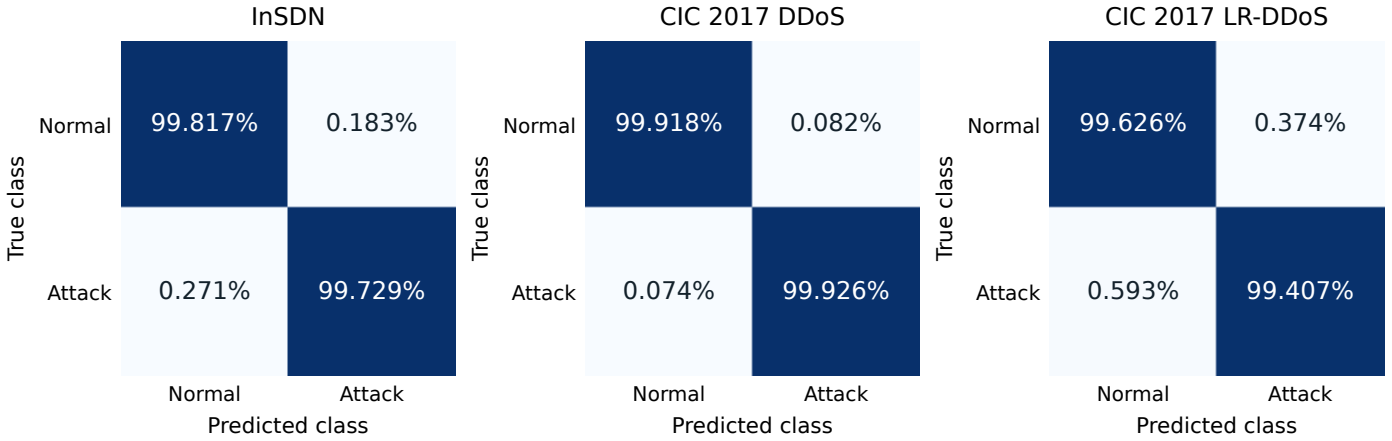


Figure 7. Mean final row-normalized confusion matrices.

The mean normal-class recognition rates are 99.817%, 99.918%, and 99.626% for InSDN, CIC 2017 DDoS, and CIC 2017 LR-DDoS, respectively. Corresponding attack-class recognition rates are 99.729%, 99.926%, and 99.407%. CIC 2017 LR-DDoS has higher final FPR and lower attack recall than the other two collections. Relative to initialization, its attack-class recognition improves while normal-class recognition decreases slightly.

7.6. Processing Time and Memory Usage

Figure 8 and Table 7 summarize the same twenty runs. Mean recorded experiment execution time is 4.984 s for InSDN, 3.267 s for CIC 2017 DDoS, and 1.319 s for CIC 2017 LR-DDoS. Mean initial classifier fitting time is 0.259, 0.220, and 0.152 s, respectively. Mean maximum sampled resident memory is 444.986, 387.744, and 293.408 MiB.

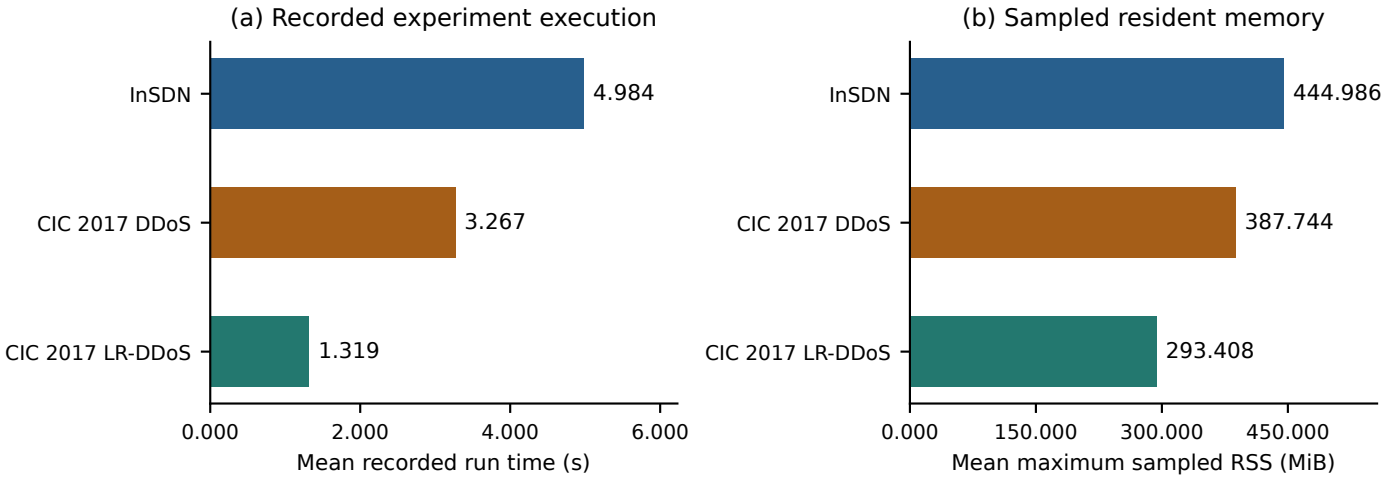


Figure 8. Comparison of mean recorded runtime and sampled memory across datasets.

Table 7. Mean initial fitting time, recorded runtime, and maximum sampled resident memory.

Dataset	Initial Fit (s)	Recorded Run (s)	Sampled RSS (MiB)
InSDN	0.259	4.984	444.986
CIC 2017 DDoS	0.220	3.267	387.744
CIC 2017 LR-DDoS	0.152	1.319	293.408

The recorded execution time includes offline evaluation and diagnostic overhead, while SOM fitting and scoring are cached separately. In an SDN deployment, the exporter must first collect packets and calculate flow features. Flow formation and feature transfer add costs beyond the measured classifier operations.

7.7. Repeated Results and Supplementary Experiments

7.7.1. Performance Changes

Table 8 summarizes initial-to-final changes in the twenty cases. Low-rate detection shows the largest mean accuracy and F1 gains. Standard deviations describe variation among these selected runs.

Table 8. Initial-to-final performance changes (mean ± SD, pp).

Dataset	Accuracy	Precision	Recall	F1	FPR
InSDN	+0.121 ± 0.078	+0.161 ± 0.294	+0.102 ± 0.237	+0.132 ± 0.091	−0.141 ± 0.287
CIC 2017 DDoS	+0.032 ± 0.016	+0.024 ± 0.038	+0.027 ± 0.041	+0.026 ± 0.013	−0.038 ± 0.059
CIC 2017 LR-DDoS	+0.523 ± 0.401	−0.033 ± 0.264	+1.424 ± 1.083	+0.704 ± 0.539	+0.025 ± 0.160

7.7.2. Classifier Controls and Component Ablations

These comparisons examine the effects of classifier retraining, pseudo-label selection, and individual filtering components.

Table 9 compares all methods on the same twenty partitions per dataset. SCX has higher mean final F1 than initial-set refitting, confidence self-training, XRDI + DT, and

the ARF adaptation on all three datasets. Initial-set refitting isolates the changed update configuration: its F1 is 99.706%, 99.924%, and 99.223%, compared with 99.770%, 99.937%, and 99.394% for SCX. The total initial-to-final gain includes an improvement associated with refitting the initial labeled set using the update configuration and should therefore not be attributed entirely to pseudo-labeled data. For CIC LR-DDoS, initial-set refitting increases mean F1 from 98.689% to 99.223%, whereas SCX reaches 99.394%, approximately 0.171 percentage points above this control. ARF results concern the pseudo-label-fed adaptation, not supervised ARF.

Table 9. Final attack-class F1 across twenty matched partitions (mean ± SD, %).

Method	InSDN	CIC DDoS	CIC LR-DDoS
Initial XGBoost	99.638 ± 0.129	99.911 ± 0.016	98.689 ± 0.556
Initial-set refit	99.706 ± 0.130	99.924 ± 0.022	99.223 ± 0.304
SCX	99.770 ± 0.104	99.937 ± 0.016	99.394 ± 0.240
Confidence self-training	99.599 ± 0.147	99.911 ± 0.016	98.689 ± 0.556
Without purity	99.568 ± 0.212	99.924 ± 0.019	99.374 ± 0.264
Without support penalty	99.772 ± 0.097	99.937 ± 0.016	99.394 ± 0.240
Without distance factor	98.105 ± 5.815	99.882 ± 0.037	99.184 ± 0.363
Without batch quantile	99.770 ± 0.104	99.937 ± 0.016	99.394 ± 0.240
XRDI + DT adaptation	99.103 ± 2.587	99.925 ± 0.025	96.916 ± 0.917
ARF + SOM labels	93.828 ± 4.003	92.893 ± 7.571	61.246 ± 8.978

7.7.3. XGBoost Confidence Self-Training

Figure 9 evaluates XGBoost self-training using the same preprocessing, batches, and initial/update classifier parameters. Pseudo-labels are the current classifier's most probable class; records enter cumulative retraining when their maximum class probability reaches 0.850 for InSDN or 0.900 for either CIC dataset.

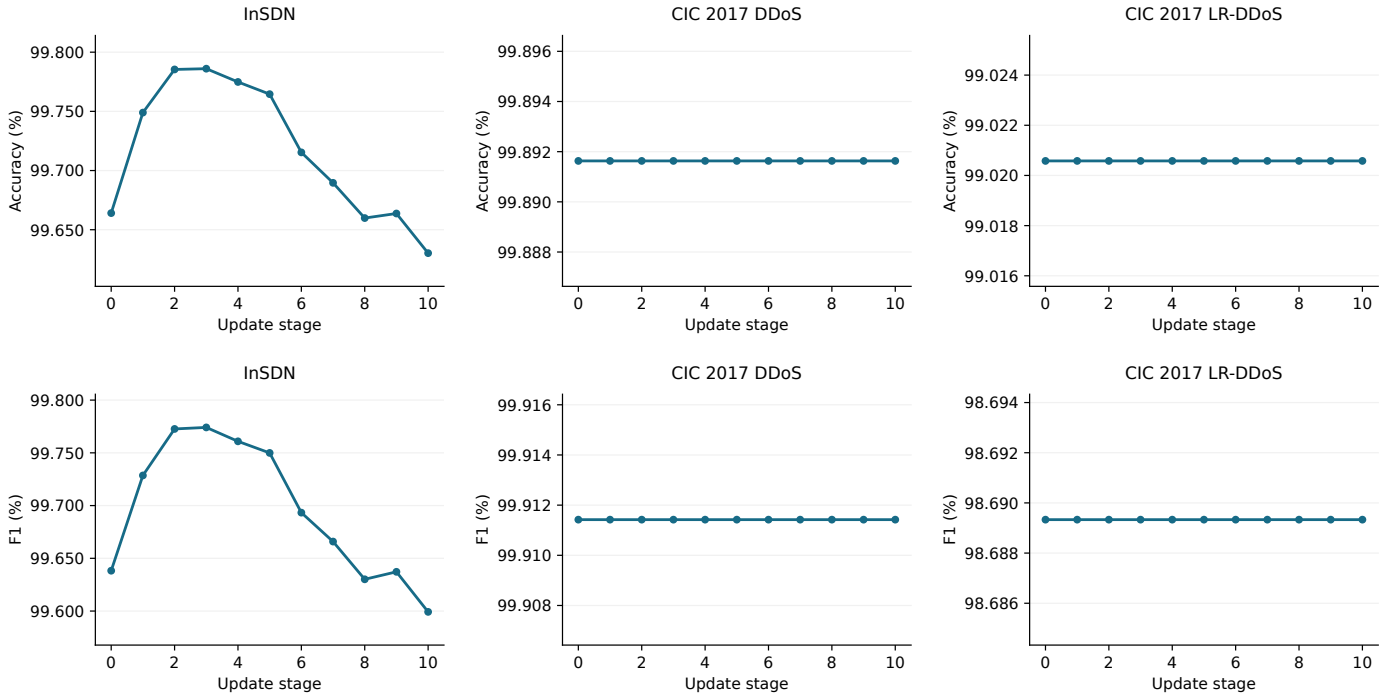


Figure 9. Mean accuracy and F1 of XGBoost confidence self-training.

InSDN improves early but subsequently declines; 15 of 20 runs update. Both CIC trajectories remain unchanged because no sample reaches 0.900, leaving the baseline

identical to the initial classifier in all runs. SCX achieves higher final mean accuracy on all three datasets and exceeds this baseline in F1 by 0.171, 0.026, and 0.704 pp, respectively (Table 9). The SOM score and classifier probability have different meanings, so equal numerical thresholds do not imply comparable admission criteria. These results therefore support an advantage over the tested configuration, rather than a general advantage over classifier-confidence self-training.

7.7.4. Pseudo-Label Admission and Reliability

The admission diagnostics in Table 10 use the same twenty partitions. Removing purity or the distance factor raises mean InSDN pseudo-label error and lowers mean final F1; removing distance also increases intake and lowers mean F1 on both CIC datasets. Support removal produces nearly unchanged F1, with a slightly higher InSDN mean, and does not demonstrate a separate performance benefit from that penalty. Removing the quantile yields identical selections and predictions in all paired comparisons because the confidence-derived distance bound is already tighter. These results support purity and distance screening in this case set, while retaining the reliability–coverage trade-off.

Table 10. Mean pseudo-label admission counts and error rates across twenty partitions.

Dataset	Gate	Accepted	Pseudo-Normal	Pseudo-Attack	Error (%)
InSDN	SCX	26,017.850	16,201.300	9816.550	0.014
InSDN	Without purity	27,207.950	16,251.100	10,956.850	0.356
InSDN	Without distance factor	50,722.100	33,585.250	17,136.850	2.968
CIC 2017 DDoS	SCX	11,543.350	392.750	11,150.600	0.001
CIC 2017 DDoS	Without purity	11,686.750	393.950	11,292.800	0.002
CIC 2017 DDoS	Without distance factor	54,730.850	16,774.050	37,956.800	0.056
CIC 2017 LR-DDoS	SCX	3117.350	54.400	3062.950	0.002
CIC 2017 LR-DDoS	Without purity	3120.450	54.900	3065.550	0.002
CIC 2017 LR-DDoS	Without distance factor	14,643.550	8635.200	6008.350	0.200

Accepted class proportions differ from the stream, especially on CIC. Rejected records are still classified by XGBoost but do not enter retraining. The gate can omit useful modes far from the initial map.

7.7.5. Sensitivity to the Initial Label Budget

Table 11 compares initial XGBoost with SCX after ten updates using the same twenty split seeds at each budget. Mean F1 decreases at 5% on all three datasets, improves at 10%, and changes only slightly at 20%. The benefit is not monotonic in the initial label budget.

Table 11. Initial and final F1 under different initial label budgets (mean ± SD, %).

Dataset	Nominal Budget	Initial XGBoost	Final SCX (Update 10)
InSDN	5%	99.618 ± 0.183	99.602 ± 0.172
InSDN	10%	99.638 ± 0.129	99.770 ± 0.104
InSDN	20%	99.766 ± 0.102	99.758 ± 0.092
CIC 2017 DDoS	5%	99.917 ± 0.018	99.911 ± 0.023
CIC 2017 DDoS	10%	99.911 ± 0.016	99.937 ± 0.016
CIC 2017 DDoS	20%	99.930 ± 0.014	99.936 ± 0.017
CIC 2017 LR-DDoS	5%	98.730 ± 0.570	98.616 ± 0.363
CIC 2017 LR-DDoS	10%	98.689 ± 0.556	99.394 ± 0.240
CIC 2017 LR-DDoS	20%	99.381 ± 0.268	99.404 ± 0.239

7.7.6. Sensitivity to Admission Parameters

The parameter study examines how the screening configuration affects validation F1 and the number of admitted records.

Across the eleven gate configurations, mean validation F1 ranges are 99.369–99.703% (InSDN), 99.793–99.826% (DDoS), and 98.061–99.222% (LR-DDoS); corresponding mean intake ranges are 15,013.333–33,309.000, 3165.000–22,922.000, and 2749.667–3702.667. None replaces the fixed main configuration.

8. Comparison

Table 12 summarizes published performance under each study's evaluation protocol and the final twenty-case SCX means from Table 6.

Table 12. Published results and SCX performance (%).

Study	Method/Evaluation Stage	Accuracy	F1
InSDN			
Han et al. (2024) [19]	XRDI + decision tree	99.970	99.980
Mehmood et al. (2025) [30]	Optimized CNN–MLP	99.980	99.980
Wang et al. (2024) [31]	MDDCC (attack/normal)	99.240	99.530
SCX (this study)	SOM + continuous XGBoost; twenty-run mean	99.785	99.770
CIC 2017 DDoS			
Han et al. (2024) [19]	XRDI + decision tree	99.950	99.950
Nguyen et al. (2024) [37]	AlexNet + SLCPL + FCM (after incremental learning)	99.260	99.340
SCX (this study)	SOM + continuous XGBoost; twenty-run mean	99.923	99.937
CIC 2017 LR-DDoS			
Widodo et al. (2025) [32]	CatBoost-RFE + MLP (7 features)	99.500	92.000
Widodo et al. (2025) [32]	CatBoost-RFE + MLP (20 features)	99.600	95.000
SCX (this study)	SOM + continuous XGBoost; twenty-run mean	99.543	99.394

SCX provides an update workflow when incoming traffic is available but additional annotations are limited. Han et al. [19] and Widodo et al. [32] emphasize feature selection for supervised classification. SCX instead selects automatically labeled records for subsequent training. Unlike the expert-label feedback in Nguyen et al.'s incremental procedure [37], SCX requires no new human annotations after initialization.

SCX separates SOM-based labeling from XGBoost updating. Admission uses local label agreement, support, and distance rather than only the classifier's prediction confidence. InSDN mean accepted-label error is 0.014% for SCX, compared with 0.356% without purity and 2.968% without the distance factor (Table 10). These results support purity and distance screening as controls on training-label noise.

Packet headers, lengths, timing, and counters provide SCX inputs without application-content inspection. This property, shared with flow-based methods such as XRDI [19], supports processing exported flow records. The update procedure is evaluated separately on DDoS and low-rate traffic, extending its evaluated scope across the three datasets.

Applicability Limits and Detection Blind Spots

The study addresses limited labels within an existing normal/attack task. Random batches do not establish temporal generalization, and Slowloris/Slowhttptest do not cover all distributed low-rate mechanisms. InSDN is SDN-derived, while CIC flow classification is not live SDN validation. The chronology and provenance limits are stated in Section 6.2.

The frozen SOM and neuron labels cannot acquire a new attack representation. Far-from-map patterns can remain excluded from retraining; no mitigation or emerging-attack degradation experiment is established here. Such records still receive XGBoost predictions, but new-family recognition is not guaranteed. Adaptive maps and reviewed labels remain future work.

This limitation does not remove the value of SOM-guided training under limited labels: within the existing task, reliable pseudo-labels can supplement the small initial labeled set without additional manual labeling.

The cache grows with accepted batches. The inactive 300,000-row fitting cap does not bound stored history, and no eviction policy is evaluated. Retaining initial records does not prove protection against catastrophic forgetting: without an old-task/new-task matrix, valid forgetting, BWT, and FWT cannot be inferred. Bounded storage and retention-aware evaluation require further study.

Identical retained feature vectors are indistinguishable to a feature-only classifier. Slow malicious requests may resemble legitimate slow clients; request completeness, application semantics, and server occupancy are unobserved. Per-flow summaries omit explicit cross-flow coordination and may lose burst-phase information relevant to TCP-targeted attacks [4,32]. Timing/size manipulation has affected other learning-based detectors [47]; SCX adversarial robustness remains untested. These are potential blind spots, not measured attack-specific failure rates.

Filtering favors reliable admitted labels over complete training-label coverage. The ablations expose this trade-off, but low admitted-label error neither covers rejected modes nor guarantees a detection gain. Verified temporal streams, bounded retention, and end-to-end SDN measurements remain future work.

9. Conclusions

This paper studied SCX for payload-independent DDoS and low-rate flow classification under limited initial labels. The inherited SOM–XGBoost workflow uses a fixed map to generate and filter pseudo-labels and accumulates accepted records for classifier retraining. The extension specifies feature provenance and implementation details, analyzes the admission rule, and evaluates with supplementary component ablations.

Final mean accuracy is 99.785%, 99.923%, and 99.543%, with mean gains of 0.121, 0.032, and 0.523 pp. Final mean F1 is 99.770%, 99.937%, and 99.394%, respectively. The largest change occurs in low-rate recall, which rises by 1.424 pp alongside a 0.025 pp increase in FPR. Supplementary ablations support analysis of label reliability and training coverage.

The additional F1 gains over initial-set refitting at the nominal 10% initial label budget support the value of SOM-filtered retraining for alleviating label scarcity within the evaluated task, without additional manual labels. The 5% results show that this benefit is not guaranteed under every initial label budget. Adaptation to new attack families and concept drift remains a separate problem requiring further investigation.

Future work will examine adaptive map representations, bounded training history, old-task retention, and capture-verified temporal evaluation. A live SDN study is also needed to measure flow formation, export, detection, and mitigation delay together.

Author Contributions: S.L. was primarily responsible for research and experimental design, coding, and writing the paper. H.F. provided guidance throughout the entire process. P.L. provided guidance on writing the paper. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding. The article processing charge (APC) is funded by Shibaura Institute of Technology. No grant number applies.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Dataset sources are given in References [17,45,46]. The authors plan public release of the implementation and experiment scripts upon acceptance, while further development continues.

Acknowledgments: During the course of this research, the authors used ChatGPT (GPT-5.6 Sol) to assist with code writing. The authors have reviewed and edited the output and assume full responsibility for the content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

BMU	Best matching unit	BWT	Backward transfer
DDoS	Distributed denial of service	DoS	Denial of service
FPR	False-positive rate	FWT	Forward transfer
IPFIX	IP Flow Information Export	RSS	Resident set size
SCX	SOM-guided Continuous XGBoost	SDN	Software-defined networking
SOM	Self-organizing map		

References

1. McKeown, N.; Anderson, T.; Balakrishnan, H.; Parulkar, G.; Peterson, L.; Rexford, J.; Shenker, S.; Turner, J. OpenFlow: Enabling Innovation in Campus Networks. *ACM SIGCOMM Comput. Commun. Rev.* **2008**, *38*, 69–74. [CrossRef]
2. Kreutz, D.; Ramos, F.M.V.; Verissimo, P.E.; Rothenberg, C.E.; Azodolmolky, S.; Uhlig, S. Software-Defined Networking: A Comprehensive Survey. *Proc. IEEE* **2015**, *103*, 14–76. [CrossRef]
3. Scott-Hayward, S.; Natarajan, S.; Sezer, S. A Survey of Security in Software Defined Networks. *IEEE Commun. Surv. Tutor.* **2016**, *18*, 623–654. [CrossRef]
4. Kuzmanovic, A.; Knightly, E.W. Low-rate TCP-targeted denial of service attacks: The shrew vs. the mice and elephants. In *Proceedings of the 2003 conference on Applications, Technologies, Architectures, and Protocols for Computer Communications*; Association for Computing Machinery: New York, NY, USA, 2003; pp. 75–86. [CrossRef]
5. Pérez-Díaz, J.A.; Amezcua Valdovinos, I.; Choo, K.K.R.; Zhu, D. A Flexible SDN-Based Architecture for Identifying and Mitigating Low-Rate DDoS Attacks Using Machine Learning. *IEEE Access* **2020**, *8*, 155859–155872. [CrossRef]
6. Wu, Z.; Xu, Q.; Wang, J.; Yue, M.; Liu, L. Low-Rate DDoS Attack Detection Based on Factorization Machine in Software Defined Network. *IEEE Access* **2020**, *8*, 17404–17418. [CrossRef]
7. El Sayed, M.S.; Le-Khac, N.A.; Azer, M.A.; Jurcut, A.D. A Flow-Based Anomaly Detection Approach With Feature Selection Method Against DDoS Attacks in SDNs. *IEEE Trans. Cogn. Commun. Netw.* **2022**, *8*, 1862–1880. [CrossRef]
8. Polat, H.; Polat, O.; Cetin, A. Detecting DDoS Attacks in Software-Defined Networks Through Feature Selection Methods and Machine Learning Models. *Sustainability* **2020**, *12*, 1035. [CrossRef]
9. van Engelen, J.E.; Hoos, H.H. A survey on semi-supervised learning. *Mach. Learn.* **2020**, *109*, 373–440. [CrossRef]
10. Parisi, G.I.; Kemker, R.; Part, J.L.; Kanan, C.; Wermter, S. Continual Lifelong Learning with Neural Networks: A Review. *Neural Netw.* **2019**, *113*, 54–71. [CrossRef] [PubMed]
11. De Lange, M.; Aljundi, R.; Masana, M.; Parisot, S.; Jia, X.; Leonardis, A.; Slabaugh, G.; Tuytelaars, T. A continual learning survey: Defying forgetting in classification tasks. *IEEE Trans. Pattern Anal. Mach. Intell.* **2022**, *44*, 3366–3385. [CrossRef] [PubMed]
12. Alashhab, A.A.; Zahid, M.S.; Isyaku, B.; Elnour, A.A.; Nagmeldin, W.; Abdelmaboud, A.; Abdullah, T.A.A.; Maiwada, U.D. Enhancing DDoS Attack Detection and Mitigation in SDN Using an Ensemble Online Machine Learning Model. *IEEE Access* **2024**, *12*, 51630–51649. [CrossRef]
13. Chen, T.; Guestrin, C. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*; Association for Computing Machinery: New York, NY, USA, 2016; pp. 785–794. [CrossRef]
14. Farhat, S.; Abdelkader, M.; Meddeb-Makhlouf, A.; Zarai, F. Evaluation of DoS/DDoS attack detection with ML techniques on CIC-IDS2017 dataset. In Proceedings of the Proc. International Conference on Information Systems Security and Privacy, Lisbon, Portugal, 22–24 February 2023; pp. 287–295.
15. Liu, S.; Fukuda, H.; Leger, P. SOM-based Continuous XGBoost Approach for DDoS Attack Detection and Mitigation in Edge SDN. In Proceedings of the 2026 29th Conference on Innovation in Clouds, Internet and Networks (ICIN), Athens, Greece, 30 March–2 April 2026; pp. 1–8. [CrossRef]
16. Liu, S.; Fukuda, H.; Leger, P. An RF-based Low Rate DDoS Attack Real-time Detection System. In Proceedings of the 2023 33rd International Telecommunication Networks and Applications Conference (ITNAC), Melbourne, Australia, 29 November–1 December 2023; pp. 304–309. [CrossRef]

17. Canadian Institute for Cybersecurity. Intrusion Detection Evaluation Dataset (CIC-IDS2017). Available online: https://www.unb.ca/cic/datasets/ids-2017.html (accessed on 5 September 2026).
18. Liu, Z.; Wang, Y.; Feng, F.; Liu, Y.; Li, Z.; Shan, Y. A DDoS Detection Method Based on Feature Engineering and Machine Learning in Software-Defined Networks. *Sensors* **2023**, *23*, 6176. [CrossRef] [PubMed]
19. Han, D.; Li, H.; Fu, X.; Zhou, S. Traffic Feature Selection and Distributed Denial of Service Attack Detection in Software-Defined Networks Based on Machine Learning. *Sensors* **2024**, *24*, 4344. [CrossRef] [PubMed]
20. Gama, J.; Žliobaitė, I.; Bifet, A.; Pechenizkiy, M.; Bouchachia, A. A survey on concept drift adaptation. *ACM Comput. Surv.* **2014**, *46*, 1–37. [CrossRef] [PubMed]
21. Saito, T.; Rehmsmeier, M. The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLoS ONE* **2015**, *10*, e0118432. [CrossRef] [PubMed]
22. Wang, M.; Lu, Y.; Qin, J. Source-Based Defense Against DDoS Attacks in SDN Based on sFlow and SOM. *IEEE Access* **2022**, *10*, 2097–2116. [CrossRef]
23. Liu, S.; Fukuda, H.; Leger, P. Real-time DDoS Attack Defense System in SDN Using LSSOM. In Proceedings of the 2023 26th Conference on Innovation in Clouds, Internet and Networks and Workshops (ICIN), Paris, France, 6–9 March 2023; pp. 69–73. [CrossRef]
24. Kiflay, A.; Tsokanos, A.; Fazlali, M.; Kirner, R. Network intrusion detection leveraging multimodal features. *Array* **2024**, *22*, 100349. [CrossRef]
25. Boulaiche, A.; Haddad, S.; Lemouari, A. A Convolutional Neural Network with Hyperparameter Tuning for Packet Payload-Based Network Intrusion Detection. *Symmetry* **2024**, *16*, 1151. [CrossRef]
26. Zhao, R.; Zhan, M.; Deng, X.; Wang, Y.; Wang, Y.; Gui, G.; Xue, Z. Yet Another Traffic Classifier: A Masked Autoencoder Based Traffic Transformer with Multi-Level Flow Representation. *Proc. AAAI Conf. Artif. Intell.* **2023**, *37*, 5420–5427. [CrossRef]
27. Doriguzzi-Corin, R.; Millar, S.; Scott-Hayward, S.; Martinez-del Rincon, J.; Siracusa, D. LUCID: A Practical, Lightweight Deep Learning Solution for DDoS Attack Detection. *IEEE Trans. Netw. Serv. Manag.* **2020**, *17*, 876–889. [CrossRef]
28. Elubeyd, H.; Yiltas-Kaplan, D. Hybrid Deep Learning Approach for Automatic DoS/DDoS Attacks Detection in Software-Defined Networks. *Appl. Sci.* **2023**, *13*, 3828. [CrossRef]
29. Nawaz, G.; Junaid, M.; Akhunzada, A.; Gani, A.; Nawazish, S.; Yaqub, A.; Ahmed, A.; Ajab, H. Detecting and Mitigating DDOS Attacks in SDNs Using Deep Neural Network. *Comput. Mater. Contin.* **2023**, *77*, 2157–2178. [CrossRef]
30. Mehmood, S.; Amin, R.; Mustafa, J.; Hussain, M.; Alsubaei, F.S.; Zakaria, M.D. Distributed Denial of Services (DDoS) Attack Detection in SDN Using Optimizer-Equipped CNN-MLP. *PLoS ONE* **2025**, *20*, e0312425. [CrossRef]
31. Wang, K.; Fu, Y.; Duan, X.; Liu, T. Detection and Mitigation of DDoS Attacks Based on Multi-Dimensional Characteristics in SDN. *Sci. Rep.* **2024**, *14*, 16421. [CrossRef] [PubMed]
32. Widodo, A.; Rochim, A.F.; Warsito, B. Enhancing Detection of Low-Rate DoS Attacks Using CatBoost-RFE and Multilayer Perceptron. *Int. J. Intell. Eng. Syst.* **2025**, *18*, 701–715. [CrossRef]
33. Wang, J.; Xu, C.; Yang, J. DELP-Net: A Differentiable Entropy Layer Pyramid Network for End-to-End Low-Rate DoS Detection. *Entropy* **2026**, *28*, 328. [CrossRef] [PubMed]
34. Gomes, H.M.; Bifet, A.; Read, J.; Barddal, J.P.; Enembreck, F.; Pfahringer, B.; Holmes, G.; Abdessalem, T. Adaptive random forests for evolving data stream classification. *Mach. Learn.* **2017**, *106*, 1469–1495. Correction in *Mach. Learn.* **2019**, *108*, 1877–1878. https://doi.org/10.1007/s10994-019-05793-3. [CrossRef]
35. Bifet, A.; Gavaldà, R. Learning from Time-Changing Data with Adaptive Windowing. In *Proceedings of the 2007 SIAM International Conference on Data Mining*; Society for Industrial and Applied Mathematics: Philadelphia, PA, USA, 2007; pp. 443–448. [CrossRef]
36. Lopez-Paz, D.; Ranzato, M. Gradient Episodic Memory for Continual Learning. In Proceedings of the Advances in Neural Information Processing Systems, Long Beach, CA, USA, 4–9 December 2017; Volume 30, pp. 6467–6476. Available online: https://proceedings.neurips.cc/paper/2017/hash/f87522788a2be2d171666752f97ddebb-Abstract.html (accessed on 5 September 2026).
37. Nguyen, T.L.; Kao, H.; Nguyen, T.T.; Horng, M.F.; Shieh, C.S. Unknown DDoS Attack Detection with Fuzzy C-Means Clustering and Spatial Location Constraint Prototype Loss. *Comput. Mater. Contin.* **2024**, *78*, 2181–2205. [CrossRef]
38. Ravi, N.; Shalinie, S.M. Learning-Driven Detection and Mitigation of DDoS Attack in IoT via SDN-Cloud Architecture. *IEEE Internet Things J.* **2020**, *7*, 3559–3570. [CrossRef]
39. Hallaji, E.; Shanmugam, V.; Razavi-Far, R.; Saif, M. A Study on Semi-Supervised Detection of DDoS Attacks under Class Imbalance. In *2025 IEEE Canadian Conference on Electrical and Computer Engineering (CCECE)*; IEEE: New York, NY, USA, 2025; pp. 507–511. [CrossRef]
40. Engelen, G.; Rimmer, V.; Joosen, W. Troubleshooting an Intrusion Detection Dataset: The CICIDS2017 Case Study. In *2021 IEEE Security and Privacy Workshops (SPW)*; IEEE: New York, NY, USA, 2021; pp. 7–12. [CrossRef]
41. Liu, L.; Engelen, G.; Lynar, T.; Essam, D.; Joosen, W. Error Prevalence in NIDS Datasets: A Case Study on CIC-IDS-2017 and CSE-CIC-IDS-2018. In *2022 IEEE Conference on Communications and Network Security (CNS)*; IEEE: New York, NY, USA, 2022; pp. 254–262. [CrossRef]

42. Arp, D.; Quiring, E.; Pendlebury, F.; Warnecke, A.; Pierazzi, F.; Wressnegger, C.; Cavallaro, L.; Rieck, K. Dos and Don'ts of Machine Learning in Computer Security. In Proceedings of the 31st USENIX Security Symposium (USENIX Security 22), Boston, MA, USA, 10–12 August 2022; pp. 3971–3988. Available online: https://www.usenix.org/conference/usenixsecurity22/presentation/arp (accessed on 5 September 2026).
43. Kohonen, T. Self-Organized Formation of Topologically Correct Feature Maps. *Biol. Cybern.* **1982**, *43*, 59–69. [CrossRef]
44. Friedman, J.H. Greedy Function Approximation: A Gradient Boosting Machine. *Ann. Stat.* **2001**, *29*, 1189–1232. [CrossRef]
45. Elsayed, M.S.; Le-Khac, N.A.; Jurcut, A.D. InSDN: A Novel SDN Intrusion Dataset. *IEEE Access* **2020**, *8*, 165263–165284. [CrossRef]
46. Sharafaldin, I.; Habibi Lashkari, A.; Ghorbani, A.A. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP 2018), Funchal, Portugal, 22–24 January 2018; pp. 108–116. [CrossRef]
47. Han, D.; Wang, Z.; Zhong, Y.; Chen, W.; Yang, J.; Lu, S.; Shi, X.; Yin, X. Evaluating and Improving Adversarial Robustness of Machine Learning-Based Network Intrusion Detectors. *IEEE J. Sel. Areas Commun.* **2021**, *39*, 2632–2647. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.